



CEBU INSTITUTE OF TECHNOLOGY
U N I V E R S I T Y

IT342-G4

System Integration and Architecture

System Design Document (SDD)

Project Title: BarberConnect

Prepared By: Cabasag, Ma. Melessa Villas

Version: 4.0.0

Date: 05/31/2026

Status: Updated Draft

REVISION HISTORY TABLE

Version	Date	Author	Changes Made	Status
1.0.0	02/02/2026	Cabasag, Ma. Melessa Villas	Initial draft	Draft
1.2.0	02/11/2026	Cabasag, Ma. Melessa Villas	Added requirements and functional requirements	Review
2.0.0	02/03/2026	Cabasag, Ma. Melessa Villas	Added Admin user functions and revised ERD	Draft
3.0.0	03/05/2026	Cabasag, Ma. Melessa Villas	Refined architecture, corrected component diagram references, updated stack, revised database schema for Firestore NoSQL, corrected API contracts	Draft
4.0.0	05/31/2026	Cabasag, Ma. Melessa Villas	Aligned SDD with current GitHub repository: Spring Boot 3.5.11 backend, React/Vite web app, Kotlin Android mobile app, Vertical Slice Architecture, Firestore, Cloudinary, JWT/OAuth2 security, updated API contracts, updated NoSQL data model, and refreshed plan.	Updated Draft

TABLE OF CONTENTS

Section	Page
Executive Summary	4
1.0 Introduction	5
2.0 Functional Requirements Specification	6
3.0 Non-Functional Requirements	11
4.0 System Architecture	13
5.0 API Contract & Communication	16
6.0 Database Design	20
7.0 UI/UX Design	25
8.0 Implementation Plan	33
9.0 Repository Alignment Notes	37

EXECUTIVE SUMMARY

Project Overview

BarberConnect is a web and mobile-based barbershop management platform that connects Customers, Barbers, and Admins in one centralized system. The system supports secure account access, appointment booking, haircut catalog management, social feed posts, comments, reactions, feedback, leave requests, income tracking, and administrative monitoring.

This updated SDD version aligns the design with the current GitHub repository structure and implementation. The major design correction is the removal of outdated Supabase/PostgreSQL references and the use of a Firestore NoSQL data model, Cloudinary-based media upload, Spring Boot REST backend, React/Vite web frontend, and Android Kotlin mobile frontend.

Updated Technology Stack

Layer	Repository-Aligned Design
Backend API	Java 17, Spring Boot 3.5.11, Maven, Spring Security, OAuth2 Client, Validation, Firebase Admin SDK 9.2.0, JWT 0.12.3, Cloudinary SDK 1.36.0
Database / BaaS	Cloud Firestore in Firebase Native mode; Firebase UID mapping for users; Firestore document collections for business data
Authentication	Email/password login, Google Sign-In/OAuth2 flow, custom JWT access token, refresh-token handling, role-based authorization
Image Storage	Cloudinary for profile, haircut style, and social feed images
Web Frontend	React 19.2, TypeScript 5.9, Vite 7.3, Axios, React Router DOM 7.13, Firebase JS SDK 12.10, Tailwind CSS 4.2, Vitest
Mobile Frontend	Android Kotlin, MVVM, Retrofit2, OkHttp3, DataStore, Firebase Auth, Google Sign-In, Glide; compile/target SDK 34, minimum SDK 26
Architecture	Vertical Slice Architecture organized by feature: auth, appointment, barber, catalog, admin, income, social, and shared services

Objectives

1. Develop a secure multi-role barber booking platform for Customer, Barber, and Admin users.
2. Implement appointment scheduling with availability, leave-day, and booking conflict validation.
3. Support haircut catalog management with base prices, style options, and media images.
4. Automate income tracking using an 80% barber share and 20% owner/platform share for completed appointments.
5. Provide social engagement through barber posts, reactions, comments, and customer feedback.
6. Provide dashboards for customers, barbers, and admins across web and mobile clients.

Scope

Included Features

- Role-based authentication and profile management for Customer, Barber, and Admin users.
- Customer appointment booking with barber selection, haircut style selection, time selection, payment method, and booking confirmation.
- Barber appointment management, haircut catalog management, leave request submission, income dashboard, and social feed posting.
- Admin monitoring of statistics, barber accounts, appointments, income, and leave request approval/decline.
- Cloudinary image upload for profile, catalog, and social feed images.
- Firestore NoSQL persistence for users, barbers, appointments, posts, comments, reactions, feedback, and income records.

Excluded Features

- Credit/debit payment gateway integration.
- SMS notification system.
- AI haircut recommendation engine.
- Multi-branch franchise management.

Version 4.0.0 Update Highlights

- Aligned the SDD with the current GitHub repository instead of the older mixed Supabase/Firebase draft.
- Updated the system stack to Spring Boot 3.5.11, Firestore, Cloudinary, React/Vite, and Android Kotlin.
- Replaced the old relational-style database wording with a Firestore NoSQL collection model.
- Added Vertical Slice Architecture mapping for backend, web, and mobile feature folders.
- Updated API contract details based on the current auth, appointment, catalog, social, leave, admin, and income services.

1.0 INTRODUCTION

1.1 Purpose

This System Design Document defines the architecture, functional requirements, non-functional requirements, API contracts, database design, UI/UX references, and implementation plan for BarberConnect. It serves as the technical design baseline for the web application, mobile application, and backend REST API.

1.2 Document Update Purpose

The previous SDD version contained outdated and mixed technology references. Version 4.0.0 updates the document to match the current implementation in the GitHub repository and corrects the design descriptions for the database, authentication, media storage, endpoint communication, and implementation architecture.

1.3 Intended Audience

- Project adviser and evaluators reviewing the system design.
- Developers maintaining the backend, web frontend, and Android mobile app.
- Testers validating functional and non-functional behavior.
- Future maintainers extending the BarberConnect platform.

1.4 Design Assumptions

- The backend REST API is the main gatekeeper for business logic, validation, and role-based access control.
- Firestore stores application data as documents and collections rather than relational SQL tables.
- Cloudinary stores uploaded images and returns image URLs saved in Firestore documents.
- The web frontend and Android mobile app communicate with the backend using JSON over HTTP/S.
- Access tokens are sent using the Authorization: Bearer <token> header; clients refresh or clear sessions when tokens expire.

2.0 FUNCTIONAL REQUIREMENTS SPECIFICATION

2.1 Project Overview

Item	Description
Project Name	BarberConnect
Domain	Service Booking, Barbershop Management, and Customer Engagement
Primary Users	Customer, Barber, Admin
Problem Statement	Manual haircut scheduling, barber availability tracking, and income tracking are inefficient, error-prone, and lack transparency.
Solution	A centralized web and mobile platform that automates booking, catalog management, customer engagement, income records, and administrative monitoring.

2.2 Core User Journeys

Journey 1: Customer Booking Flow

1. Customer registers or logs in using email/password or supported Google Sign-In flow.
2. Customer browses barber profiles, haircut styles, social feed posts, and available services.
3. Customer selects a barber, haircut style, optional styling options, appointment date/time, and payment method.
4. System validates availability, leave requests, duplicate schedules, and selected service details.
5. System creates an appointment document and displays it in the customer appointment list and barber schedule.

6. After completion, customer may submit rating and feedback for the appointment.

Journey 2: Barber Management Flow

1. Barber logs in and accesses the barber dashboard.
2. Barber views upcoming appointments, appointment status, and customer details.
3. Barber creates and updates haircut styles, prices, duration, options, and images.
4. Barber creates social feed posts and replies to comments or feedback.
5. Barber submits leave requests for admin approval and manages availability.
6. Barber views income records generated from completed appointments.

Journey 3: Admin Monitoring Flow

1. Admin logs in with an authorized ADMIN role.
2. Admin views platform statistics such as total revenue, total appointments, active barbers, and registered customers.
3. Admin creates or manages barber accounts and monitors barber availability.
4. Admin reviews and approves or declines barber leave requests.
5. Admin monitors appointment, income, social feed, and feedback activity for platform oversight.

2.3 Feature List (MoSCoW)

Priority	Features
MUST HAVE	User registration/login, role-based access control, customer booking, appointment management, haircut catalog, income tracking, admin dashboard, customer/barber profiles, feedback.
SHOULD HAVE	Google Sign-In, social feed posts, comments, reactions, leave request management, dashboard statistics, Cloudinary image upload.
COULD HAVE	Advanced analytics charts, barber ranking, automatic notifications, richer moderation tools.
WON'T HAVE	Credit/debit payment gateway, SMS integration, AI haircut recommendation, multi-branch support.

2.4 Detailed Feature Specifications

Feature	Specification
User Authentication	Registration, login, validation, token refresh, logout/session clearing, Google Sign-In support, role assignment for CUSTOMER, BARBER, and ADMIN.
Haircut & Service Catalog	Barbers manage haircut style records, descriptions, base prices, duration, image URL, and style options with additional prices/time. Customers and admins can view catalog data.
Appointment Scheduling	Customers book appointments by selecting barber, service, date/time, options, and payment method. Backend validates conflicts, status, and ownership.
Payment & Income Tracking	When appointments are completed/paid, records store gross amount, platform fee/owner share, barber net amount, payment method, status, and timestamp.
Social Feed & Feedback	Barbers create posts with images/captions. Customers view posts, add comments/reactions, and submit appointment feedback with ratings and comments.
Leave Requests	Barbers submit leave requests. Admin approves or declines requests, affecting availability and booking validation.
Admin Monitoring	Admin views dashboard statistics, manages barbers, monitors appointments/income, and moderates platform activity.

2.5 Acceptance Criteria

Acceptance Criteria	Expected Result
AC-1 Successful Registration	Given a new user enters valid details and a strong password, when the user submits registration, then the system creates the user account, stores a user profile document, assigns the selected role, and redirects the user to the proper dashboard.
AC-2 Successful Login	Given an existing user enters valid credentials, when login succeeds, then the client stores the access token and refresh token, then loads the dashboard according to the user role.
AC-3 Appointment Booking	Given a logged-in customer selects a barber, service, date/time, options, and payment method, when the booking is valid, then the system creates an appointment and shows it in both customer and barber appointment views.
AC-4 Booking Conflict Prevention	Given a selected time slot is already booked or the barber is on approved leave, when the customer attempts booking, then the system rejects the booking with a clear validation message.
AC-5 Barber Leave Request	Given a barber submits a leave request, when admin approves it, then the date is marked unavailable for future customer bookings.
AC-6 Income Tracking	Given an appointment is completed and paid, when the income record is generated, then the system stores the 80% barber share and 20% owner/platform share.
AC-7 Admin Dashboard	Given an admin opens the dashboard, then the system displays platform statistics such as appointments, revenue, active barbers, and customer counts.

3.0 NON-FUNCTIONAL REQUIREMENTS

3.1 Performance Requirements

- API response time should be within 2 seconds for common dashboard, booking, and catalog requests under normal school-demo load.
- Web pages should load within 3 seconds on a stable broadband connection after initial asset caching.
- Mobile app screens should load within 3 seconds for common flows such as login, dashboard, and booking.
- Firestore reads/writes and Cloudinary image upload workflows should show clear loading states when latency is present.

3.2 Security Requirements

- All production network traffic must use HTTPS/TLS.
- Protected API endpoints require Authorization: Bearer <JWT> and server-side validation through Spring Security.
- Role-based access control must restrict CUSTOMER, BARBER, and ADMIN features at both API and UI levels.
- Secrets such as Firebase service account keys, JWT secret, Cloudinary keys, and Google service files must not be committed to GitHub.
- Passwords must not be stored in Firestore as plain text. Authentication is handled through the configured authentication service and backend token flow.

3.3 Compatibility Requirements

- Web application supports current versions of Chrome, Edge, Firefox, and Safari.

- Android application supports API Level 26+ with compile/target SDK 34.
- Responsive layouts should support mobile, tablet, and desktop screen widths.
- Backend requires Java 17 and runs with Maven/Spring Boot configuration.

3.4 Usability Requirements

- The customer booking flow should be understandable and completable within a few minutes.
- Forms must display meaningful error messages for invalid fields, unavailable slots, and failed login attempts.
- Dashboards should clearly separate Customer, Barber, and Admin actions.
- Mobile views should maintain visible text, readable controls, and accessible buttons on common Android devices.

3.5 Reliability Requirements

- Booking creation and income updates must be server-controlled to avoid inconsistent client-side calculations.
- The client should recover from expired access tokens by attempting refresh-token flow or requiring re-login.
- Critical user actions should return clear success or failure states.

3.6 Scalability and Maintainability

- Firestore collections and indexes should support growth in users, appointments, posts, and income records.
- Vertical Slice Architecture keeps each feature modular, making new feature maintenance easier.
- Reusable services on web and mobile should centralize API calls and token handling.

3.7 Testability

- Backend tests use JUnit/Spring Security test dependencies as configured in the backend project.
- Frontend tests use Vitest and Testing Library dependencies.
- Manual integration tests must cover login, role routing, booking, catalog, social feed, leave request, and admin dashboard flows.

4.0 SYSTEM ARCHITECTURE

4.1 Component Diagram

The updated component diagram shows the current repository-aligned design. The system uses two client applications, a Spring Boot REST API, feature modules organized through Vertical Slice Architecture, and external services for authentication, Firestore data persistence, and Cloudinary media storage.

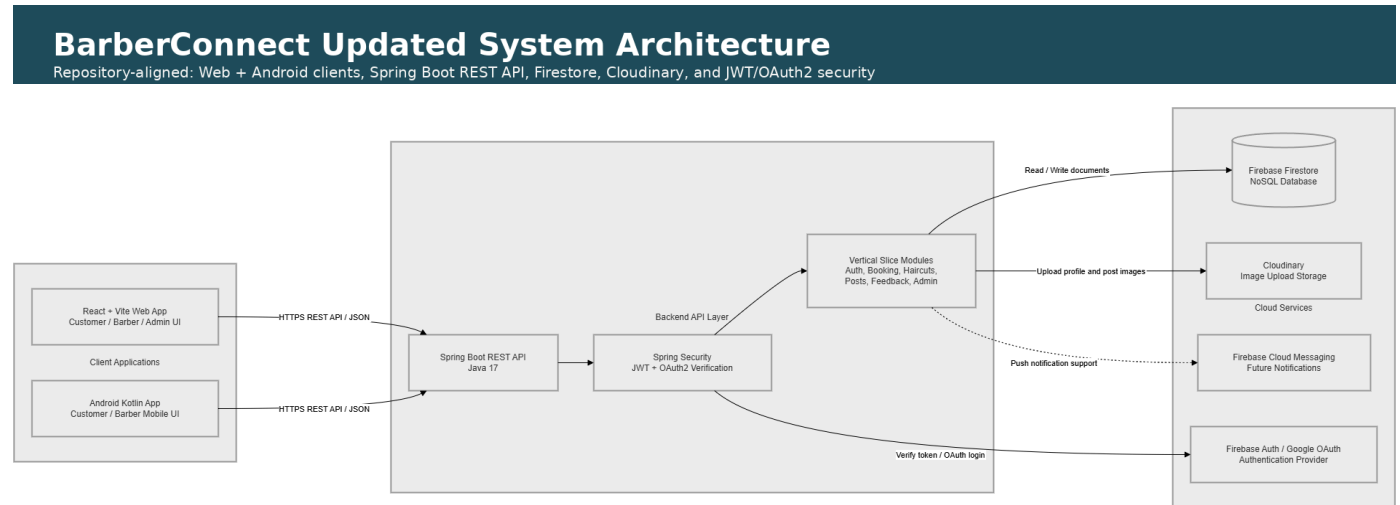


Figure 1. Updated BarberConnect component architecture.

4.2 Vertical Slice Architecture

Slice	Backend Responsibility	Web Responsibility	Mobile Responsibility
shared	FirebaseService, CloudinaryService, security utilities, common API response structures.	API client configuration, shared components, token helpers.	RetrofitClient, ApiService, TokenManager, shared models.
auth	AuthService, User model, JWT/OAuth2 handling, role validation.	Login/register pages, authService, protected routes.	LoginActivity, RegisterActivity, AuthViewModel, Firebase/Google Sign-In.
appointment	AppointmentService, appointment model, booking validation.	Appointment service, booking pages, appointments panel.	Booking and appointment viewing screens.
barber	BarberService, BarberProfile, leave request handling.	Barber dashboard panels and barber profile pages.	Barber/customer dashboard views as supported.
catalog	HaircutStyleService, HaircutStyle, StyleOption.	Catalog management and haircut style service.	Catalog/booking display through API models.
admin	AdminService, dashboard statistics, barber creation, leave approval.	Admin dashboard pages and adminService.	Admin functions if supported by mobile scope.
income	IncomeRecord model and income query logic.	Income dashboard display.	Income data display where applicable.
social	Post, Comment, Feedback, Reaction features.	Social feed, comments, reactions, feedback UI.	Feed viewing and interaction where supported.

4.3 Deployment View

Component	Deployment Description
Backend	Spring Boot application served locally during development or deployed to a Java-capable hosting platform such as Railway.
Web	React/Vite application served locally with Vite during development and deployable to static hosting such as Vercel/Netlify.
Mobile	Android APK installed on an emulator or physical Android device. Real device testing must point BASE_URL to the computer local IP.
External Services	Firebase project for Firestore/Auth, Cloudinary for image uploads, Google services configuration for web/mobile sign-in.

5.0 API CONTRACT & COMMUNICATION

5.1 API Standards

Standard	Description
Base URL	http(s)://<server-hostname>/api/v1. Web uses VITE_API_BASE_URL. Mobile uses BASE_URL from Gradle BuildConfig.
Format	JSON request and response bodies.
Authentication Header	Authorization: Bearer <JWT access token>.
Token Refresh	If a protected request returns 401 and a refresh token exists, the web client calls /auth/refresh, stores the new token, and retries the original request.
Error Handling	Standard HTTP status codes with a consistent error object containing code, message, details, and timestamp where applicable.

5.2 Standard Response Envelope

All API responses should follow a consistent response structure to simplify frontend and mobile error handling:

```
{
  "success": true,
  "data": { },
  "error": null,
  "timestamp": "2026-05-31T10:30:00Z"
}
```

5.3 Endpoint Specifications

Method	Endpoint	Purpose
POST	/auth/register	Create user account/profile with firstName, lastName, email, password, and role.
POST	/auth/login	Authenticate credentials and return user data, access token, refresh token, and role.
POST	/auth/refresh	Issue a new access token using refresh token.
GET	/auth/validate	Validate current token/session and return user context.
POST	/appointments/book	Create a customer appointment with barberProfileId, haircutStyleId, date/time, selectedOptionIds, totalPrice, and paymentMethod.
GET	/appointments/customer/{customerId}	Retrieve customer appointment history.
GET	/appointments/barber/{barberId}	Retrieve barber schedule/appointments.
PUT	/appointments/{id}/status	Update appointment status such as confirmed, completed, cancelled, or no-show.
GET	/haircuts/barber/{barberProfileId}	Retrieve all haircut styles for a barber.
GET	/haircuts/{id}/options	Retrieve style options for a selected haircut style.
POST/PUT/DELETE	/haircuts	Create, update, or soft-delete haircut styles and options.
POST	/posts	Create barber post with caption/content and image URL.
POST	/posts/{postId}/comments	Add a comment to a social feed post.
POST	/posts/{postId}/reactions	Add or update a reaction for a social feed post.
POST/GET	/feedback	Create and retrieve customer feedback and barber replies.
POST	/barbers/{id}/leave-request	Submit a barber leave request.
GET/POST	/admin/leave-requests	View and approve/decline barber leave requests.
GET	/admin/dashboard-stats	Retrieve admin dashboard statistics.
POST	/admin/barbers/create	Create a barber account/profile from admin dashboard.
GET	/barbers/{id}/income	Retrieve income records for a barber.

5.4 Booking Request Example

```
{
  "customerId": "uid_customer_001",
  "barberProfileId": "barber_profile_001",
  "haircutStyleId": "haircut_style_001",
  "appointmentDateTime": "2026-05-31T14:00:00",
  "selectedOptionIds": ["style_option_001"],
}
```

```
"totalPrice": 250.00,  
"paymentMethod": "CASH"  
}
```

5.5 Common Error Codes

Code	Description
AUTH-001	Invalid email, password, or login credentials.
AUTH-002	Missing, expired, or invalid token.
AUTH-003	User role is not allowed to access the requested resource.
VALID-001	Required input is missing or invalid.
BOOKING-001	Selected appointment schedule is unavailable or conflicts with an existing booking.
DB-001	Requested Firestore document was not found.
MEDIA-001	Cloudinary image upload failed.
SYSTEM-001	Unexpected backend error.

6.0 DATABASE DESIGN

6.1 Firestore NoSQL Data Model ERD

The current implementation uses Cloud Firestore as the primary application data store. Unlike the earlier relational table wording, this design stores data as Firestore collections and documents. Relationships are maintained through document IDs and logical reference fields such as `user_id`, `customer_id`, `barber_profile_id`, `appointment_id`, `post_id`, and `haircut_style_id`.

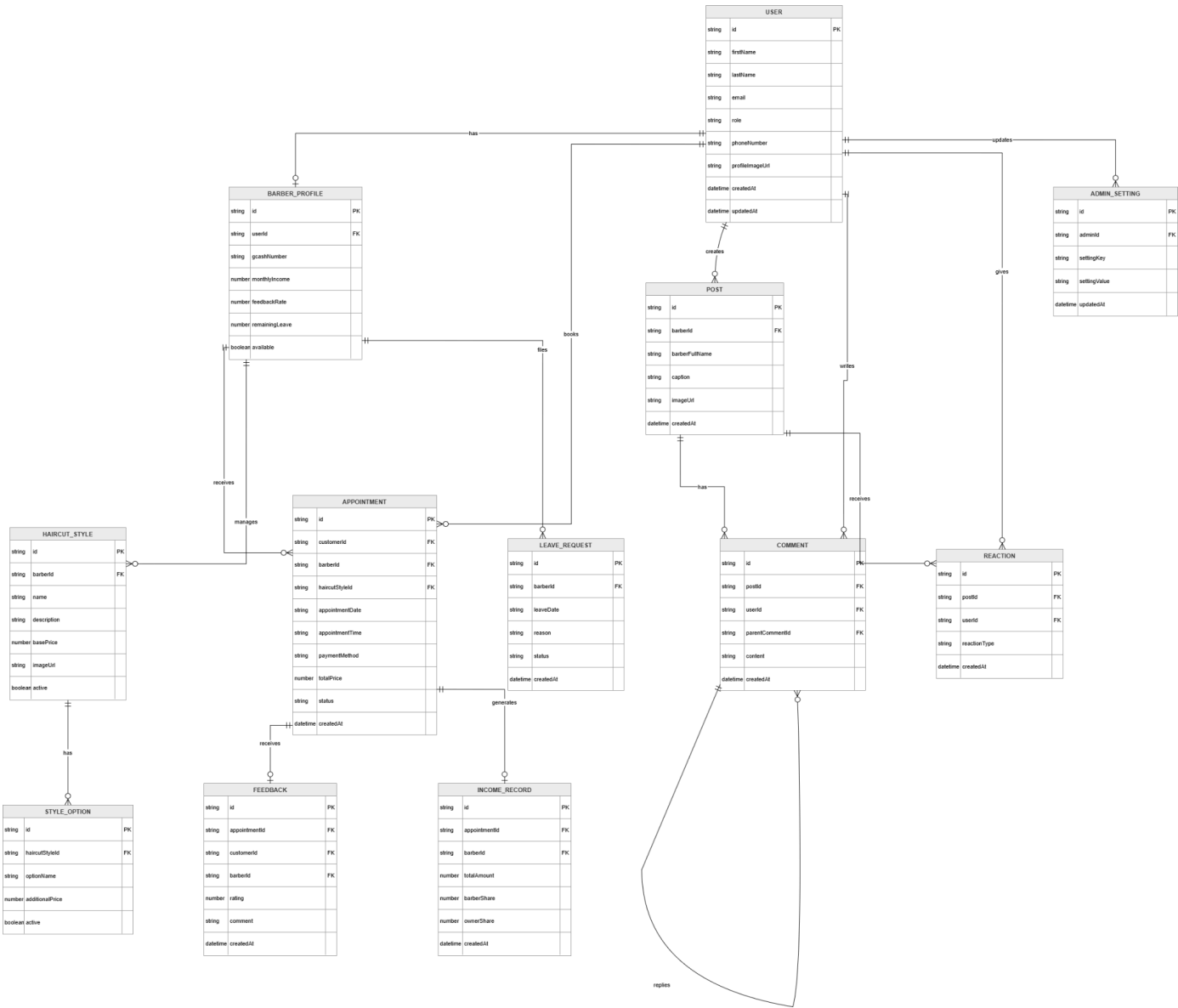


Figure 2. Updated Firestore NoSQL data model and logical relationships.

6.2 Collection Structure Summary

Collection	Key Fields	Purpose
users	user_id/firebaseUid, email, firstName, lastName, phoneNumber, role, isActive, profileImageUrl	Stores account profile and role information.
barber_profiles	barber_profile_id, user_id, bio, experience, specialties, gcashNumber, isAvailable	Stores barber-specific information linked to a user account.

haircut_styles	haircut_style_id, barber_profile_id, name, description, basePrice, durationMinutes, imageUrl, isActive	Stores barber service catalog items.
style_options	style_option_id, haircut_style_id, name, description, additionalPrice, additionalTimeMinutes, isActive	Stores optional add-ons for haircut styles.
appointments	appointment_id, customer_id, barber_profile_id, haircut_style_id, appointmentDateTime, selectedOptionIds, totalPrice, status, paymentMethod, paymentStatus	Stores booking details and status.
posts	post_id, barber_profile_id, caption/content, imageUrl, createdAt, updatedAt	Stores barber social feed posts.
comments	comment_id, post_id, user_id, commentText, createdAt	Stores comments on posts.
reactions	reaction_id, post_id, user_id, type, createdAt	Stores likes/loves/other reaction types.
feedback	feedback_id, appointment_id, customer_id, barber_profile_id, rating, comment, barberReply, createdAt	Stores post-service customer feedback.
income_records	income_record_id, appointment_id, barber_profile_id, grossAmount/amount, platformFee, netAmount, paymentMethod, paymentStatus, recordedAt	Stores income distribution records.
leave_requests	leave_request_id, barber_profile_id, startDate, endDate, reason, status, approvedBy, reviewedAt	Stores barber leave requests and admin decision.
admin_settings	settings_id, policyKey, policyValue, minPrice, maxPrice, updatedAt	Stores system-level settings and pricing policies.

6.3 Logical Relationships

- One user with role BARBER may have one barber_profile document.
- One barber_profile can own many haircut_styles, posts, appointments, income_records, and leave_requests.
- One haircut_style can have many style_options.
- One customer user can create many appointments.
- One appointment references one customer, one barber_profile, and one haircut_style; it may also reference multiple selectedOptionIds.
- One completed appointment can produce one feedback record and one or more income_records depending on business implementation.
- One post can have many comments and reactions.
- Admin users manage leave_requests and system-level settings.

6.4 Firestore Index and Validation Notes

- Appointments should be indexed by customer_id, barber_profile_id, appointmentDateTime, and status for dashboard queries.
- Posts should be indexed by createdAt and barber_profile_id for feed display.
- Income records should be indexed by barber_profile_id and recordedAt for monthly dashboard filtering.
- Leave requests should be indexed by barber_profile_id and status for admin approval workflows.
- Backend services must validate role ownership before writing or reading protected documents.

7.0 UI/UX DESIGN

The wireframes below are maintained from the original SDD and are now connected to the updated architecture and feature list. They represent the main web and mobile screens used by Customer, Barber, and Admin roles.

7.1 Web Application Wireframes

Login Page



Wireframe: Login Page.

Register Page



The Register Page wireframe features a dark blue header with the title "Register". Below the header is a large white container. At the top of this container is a small square icon with a smaller square inside, positioned above a dark blue rectangular button. Below the button is a vertical stack of form fields, each preceded by a dark blue label bar. The fields include: a name field, an email field with an envelope icon, a phone number field with a telephone icon, and two password fields with lock icons and toggle visibility icons. At the bottom of the form stack is a dark blue button with a light blue bar. Below the button is a light blue bar with a dark blue bar.

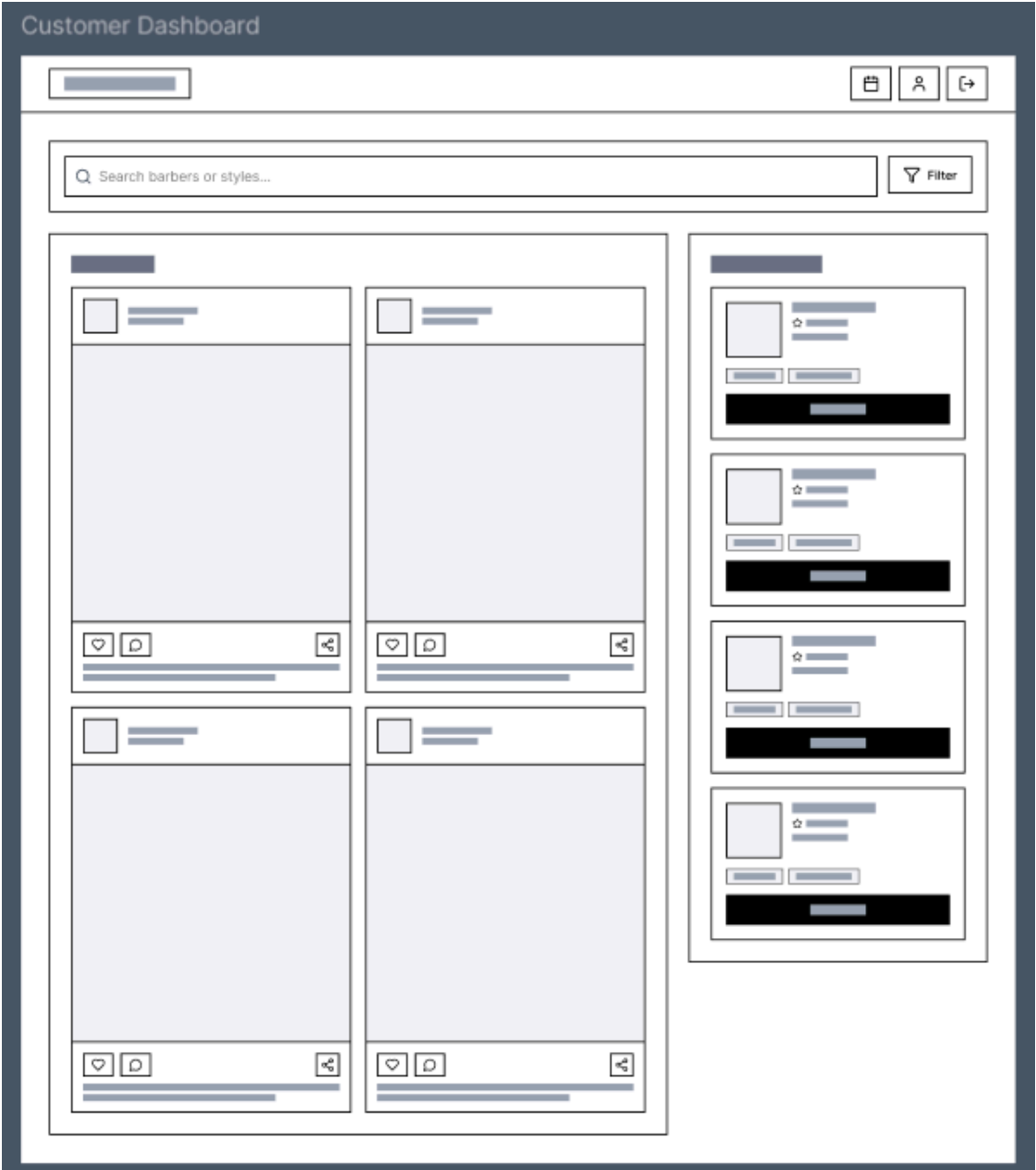
Wireframe: Register Page.

Forgot Password Page



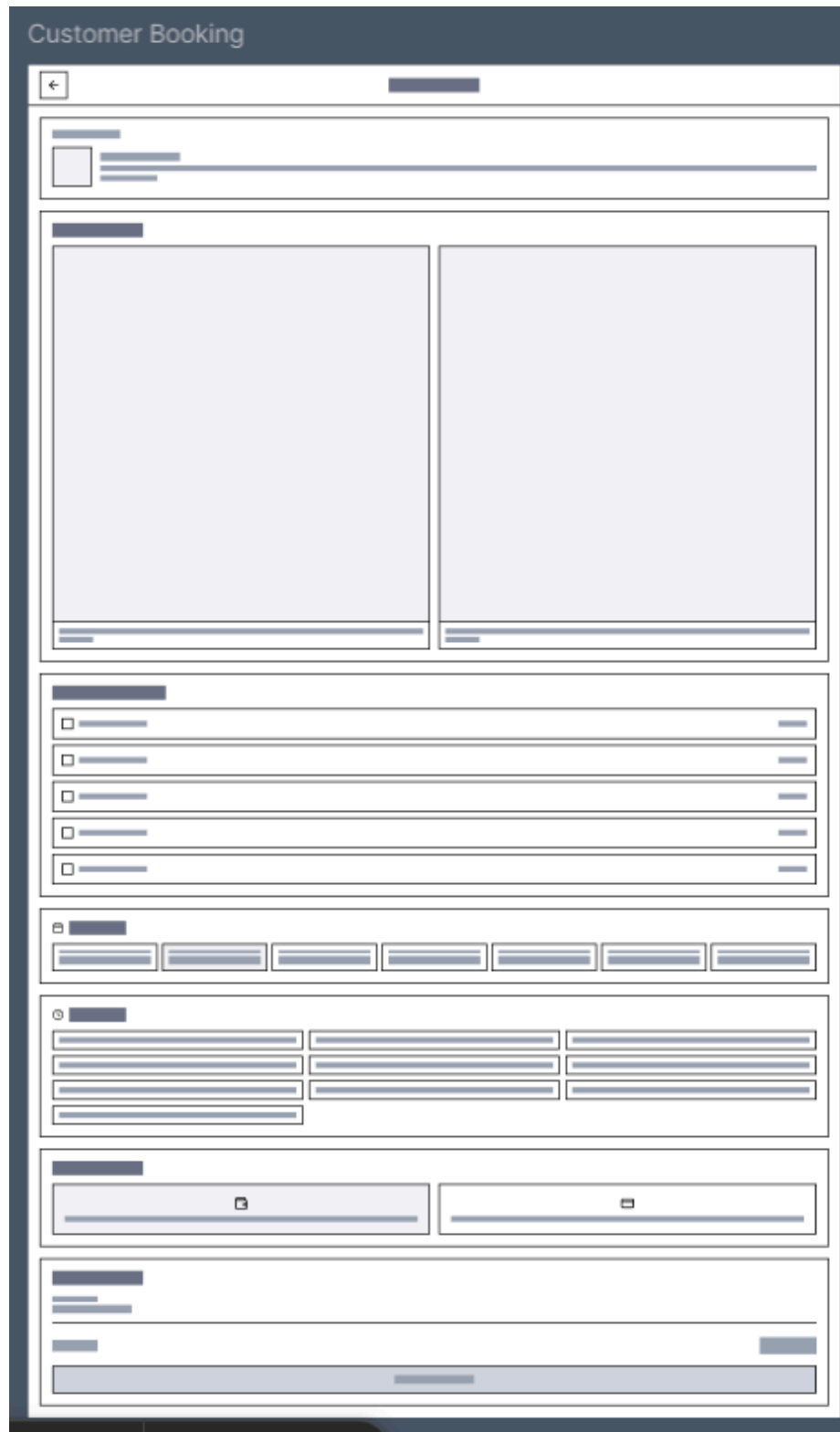
The Forgot Password Page wireframe features a dark blue header with the title "Forgot Password". Below the header is a large white container. At the top of this container is a small square icon with a smaller square inside, positioned above a dark blue rectangular button. Below the button is a vertical stack of form fields, each preceded by a dark blue label bar. The fields include: a name field, an email field with an envelope icon, and a password field with a lock icon and a toggle visibility icon. At the bottom of the form stack is a dark blue button with a light blue bar. Below the button is a light blue bar with a dark blue bar and a left-pointing arrow.

Customer Dashboard



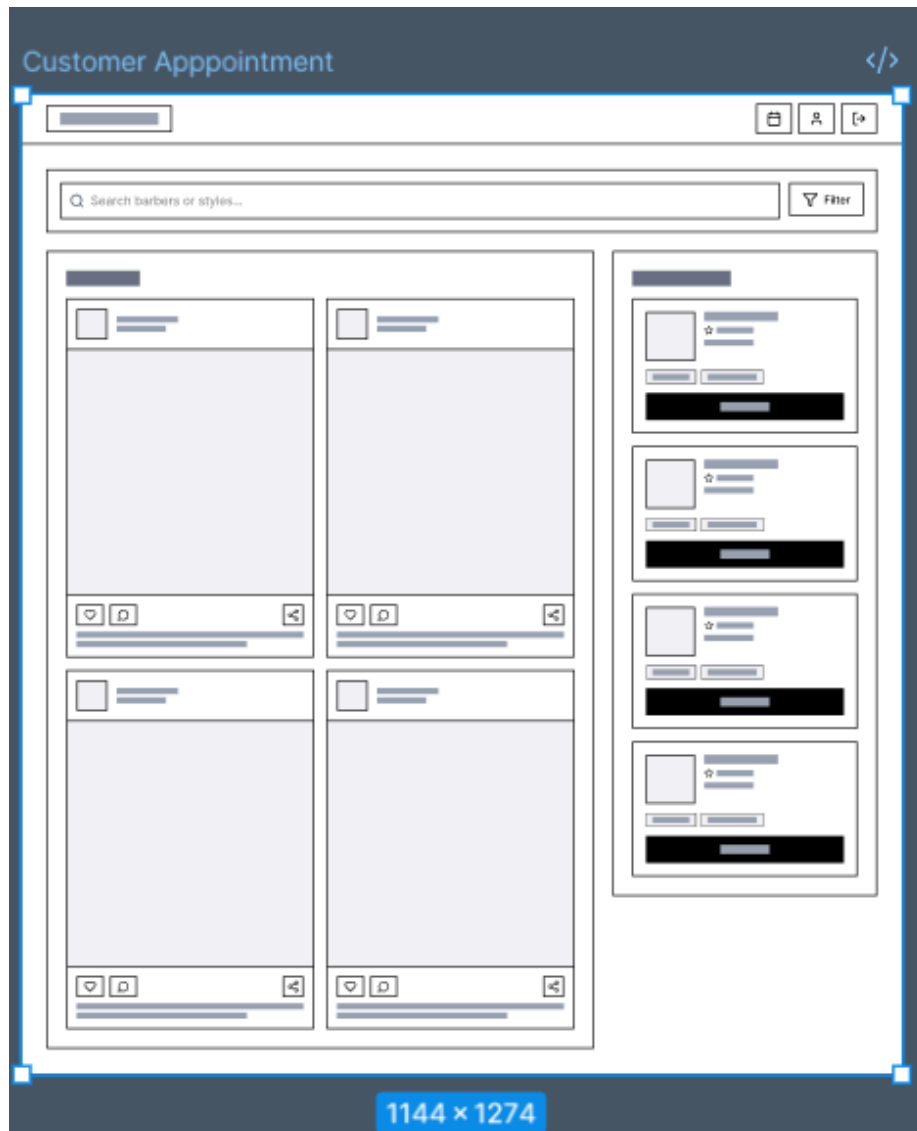
Wireframe: Customer Dashboard.

Customer Booking



Wireframe: Customer Booking.

Customer Appointment / My Bookings



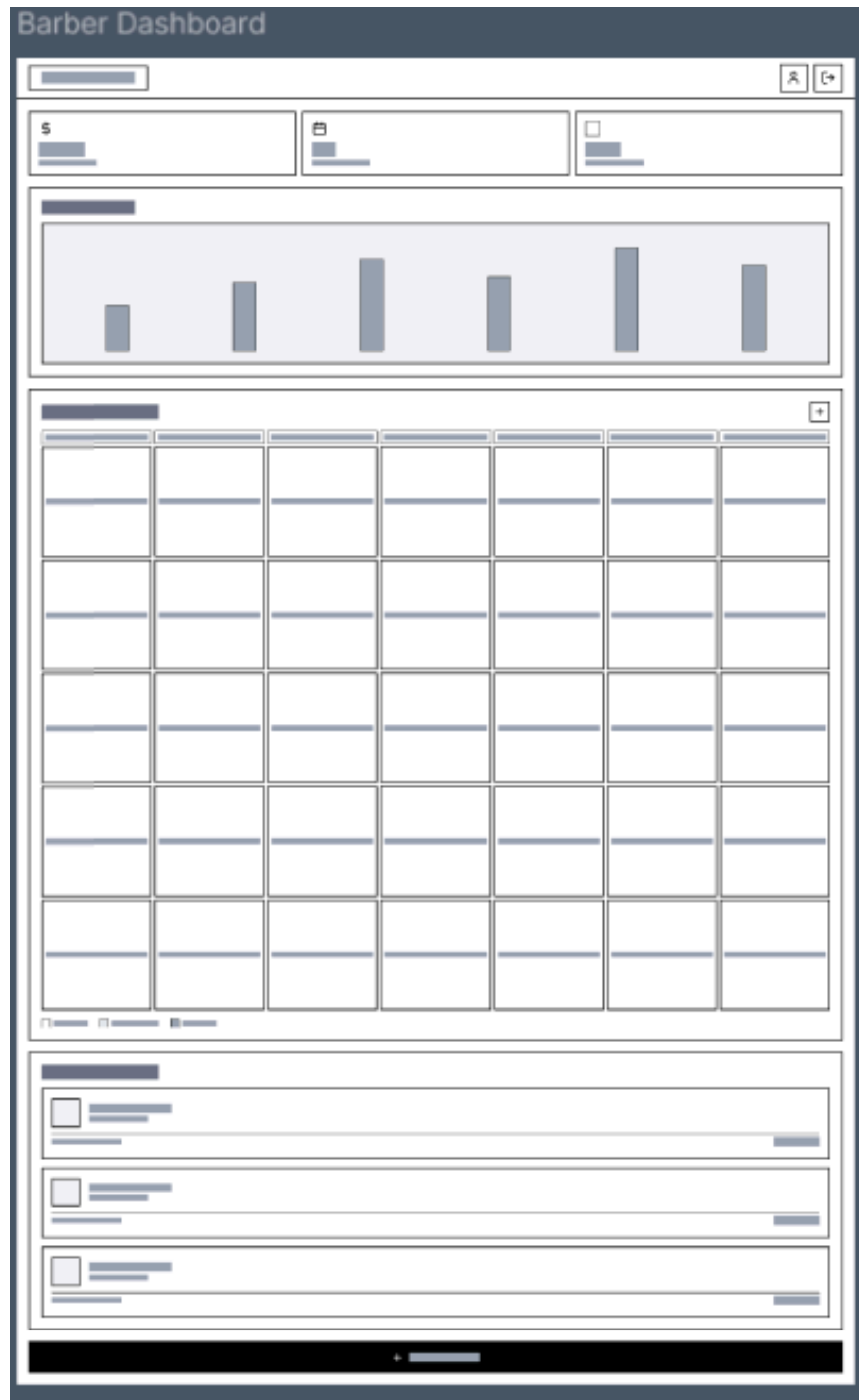
Wireframe: Customer Appointment / My Bookings.

Customer Profile Page



Wireframe: Customer Profile Page.

Barber Dashboard



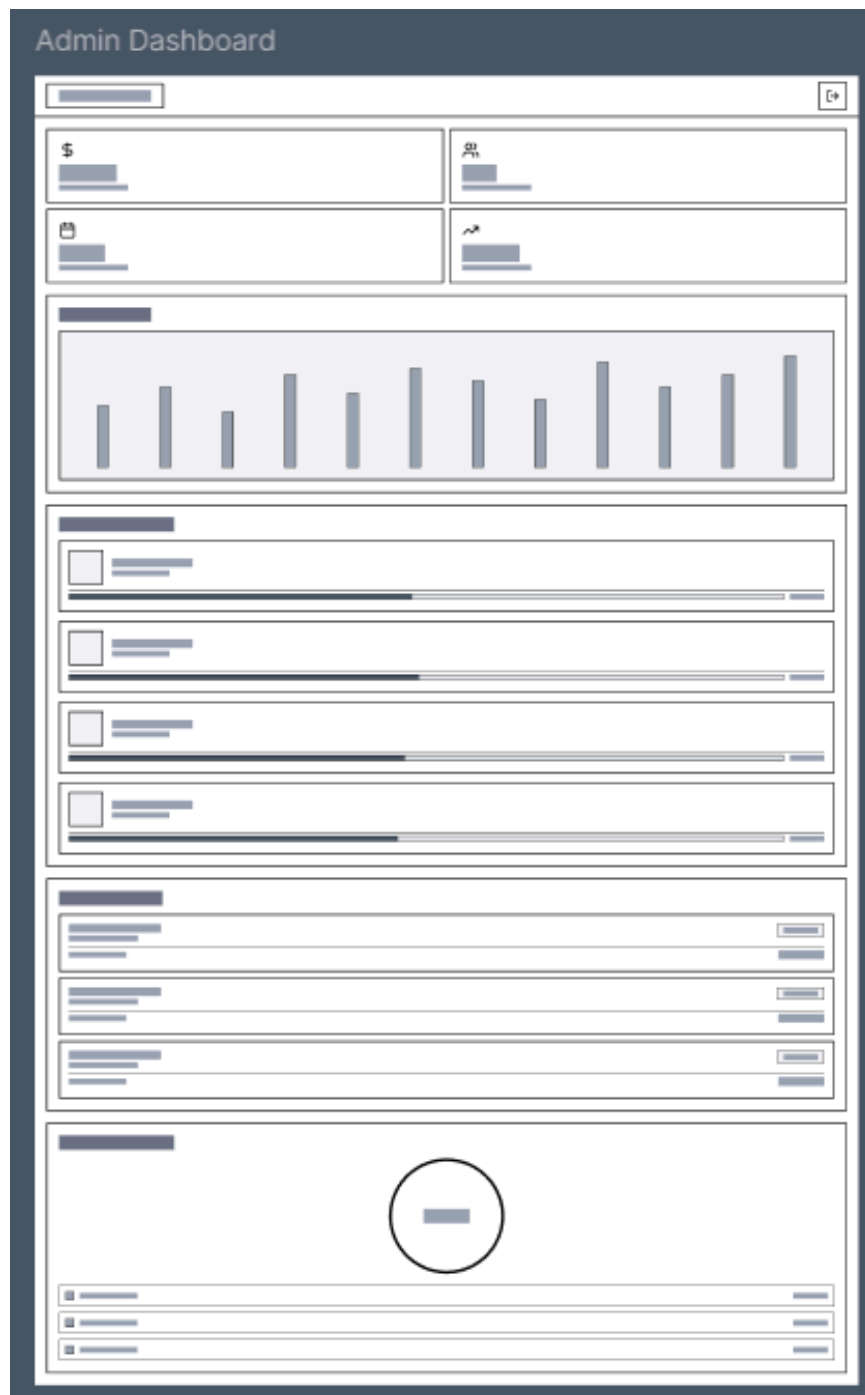
Wireframe: Barber Dashboard.

Barber Profile Page



Wireframe: Barber Profile Page.

Admin Dashboard Page



Wireframe: Admin Dashboard Page.

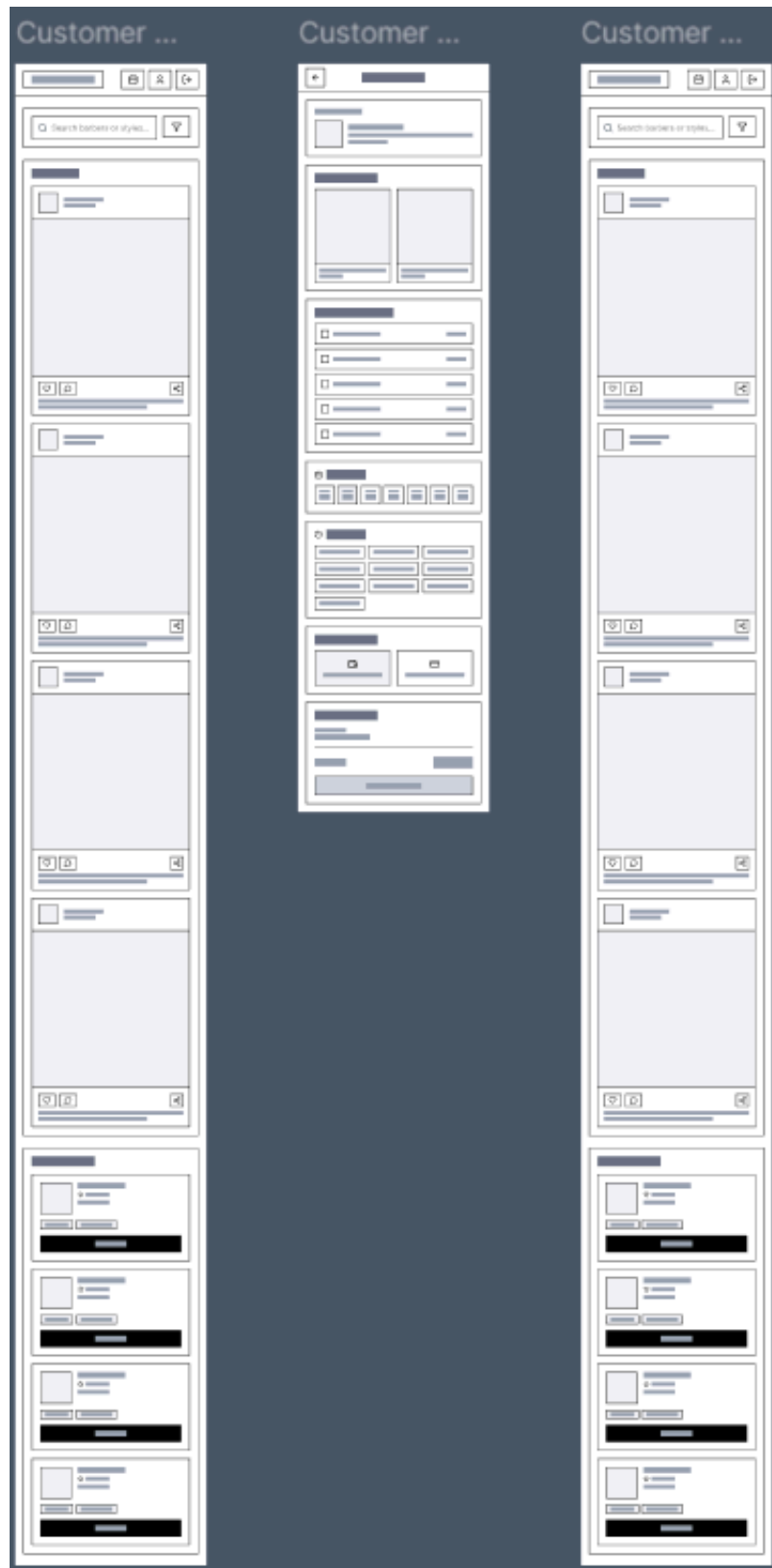
7.2 Mobile Application Wireframes

Mobile Authentication Screens



Wireframes: mobile login, register, and forgot password screens.

Customer Dashboard, Booking, and Appointment



Wireframe: Customer Dashboard, Booking, and Appointment.

Customer Profile, Barber Dashboard, and Barber Profile



Wireframe: Customer Profile, Barber Dashboard, and Barber Profile.

Admin Dashboard



Wireframe: Admin Dashboard.

7.3 UI/UX Design Rules

- Customer, Barber, and Admin dashboards must use clear role-based navigation and prevent users from accessing screens outside their role.
- Booking pages must clearly display selected barber, service, options, date/time, payment method, total price, and confirmation state.

- Forms must show inline validation for missing fields, invalid email, weak password, unavailable time slots, and failed uploads.
- Mobile screens must use visible text colors, readable buttons, and scrollable containers for small-screen devices.
- Social feed cards must show barber name/avatar, post image, caption, comments, and reaction controls clearly.

7.4 Role-Based Screen Mapping

Role	Main Screens	Primary Purpose
Customer	Login/Register, Customer Dashboard, Booking Page, My Bookings, Profile, Social Feed	Book services, view barber posts, manage appointments and profile.
Barber	Barber Dashboard, Catalog Management, Schedule/Appointments, Profile, Posts, Income, Leave Request	Manage services, bookings, social posts, income, and availability.
Admin	Admin Dashboard, Barber Management, Leave Requests, Statistics, Monitoring Views	Oversee platform users, appointments, income summaries, and leave approvals.

8.0 IMPLEMENTATION PLAN

8.1 Updated Project Timeline

Period	Phase	Deliverables
Week 1-2	Planning & Design	Finalize requirements, SDD, database model, API contracts, diagrams, and test plan.
Week 3-4	Backend Development	Implement auth, roles, appointments, catalog, barber profile, admin, income, leave requests, social feed, and validation.
Week 5-6	Web Application	Implement React/Vite pages, services, protected routes, dashboards, booking flow, catalog UI, social feed, and admin panels.
Week 7-8	Android Mobile Application	Implement login/register, dashboard, booking, appointment viewing, profile, Retrofit client, token management, and device testing.
Week 9	Integration & Testing	End-to-end role testing, booking conflict testing, media upload testing, Firestore data validation, and regression testing.
Week 10	Deployment & Finalization	Prepare backend/web/mobile deployment, APK generation, documentation update, final test evidence, and presentation readiness.

8.2 Milestones

- M1: Updated SDD, diagrams, API contracts, and Firestore model approved.
- M2: Backend API supports authentication, roles, booking, catalog, social, leave, income, and admin features.
- M3: Web application supports main Customer, Barber, and Admin flows.
- M4: Android mobile application supports authentication, dashboard, booking, and appointment viewing.
- M5: Full system passes end-to-end integration testing and is ready for final presentation.

8.3 Critical Path

1. Authentication and role routing.
2. Booking conflict validation and appointment lifecycle.
3. Haircut catalog and price calculation.
4. Income record generation for completed appointments.
5. Admin dashboard statistics and leave approval.
6. Cross-platform token handling and API base URL configuration.

8.4 Risk Mitigation

Risk	Mitigation
Secret leakage	Keep Firebase service account, JWT secret, Cloudinary keys, and google-services.json outside GitHub and load them through environment/config files.
Booking conflicts	Validate availability and duplicate schedule conditions in the backend, not only in the client.
Mobile network errors	Use the correct BASE_URL for emulator or physical device and confirm backend is reachable on the same network.
Data inconsistency	Use backend-controlled writes for appointments, income records, and role-protected admin actions.
UI regressions	Run web/mobile role-based manual checks after each major UI update.

9.0 REPOSITORY ALIGNMENT NOTES

This section summarizes the concrete updates made from the previous SDD version to the current repository-aligned version.

Area	Update Made
Database	Removed Supabase/PostgreSQL references and aligned the document with Cloud Firestore NoSQL.
Image Storage	Changed Firebase Storage wording to Cloudinary, matching the repository README and backend dependency.
Backend Version	Updated backend stack to Java 17 and Spring Boot 3.5.11 with Firebase Admin SDK, Cloudinary SDK, OAuth2 Client, and JWT.
Frontend Version	Updated actual web package details to React 19.2, TypeScript 5.9, Vite 7.3, Axios, Firebase JS SDK, Tailwind CSS, and Vitest.
Mobile Version	Updated Android SDK details to min SDK 26 and compile/target SDK 34 with Retrofit, OkHttp, DataStore, Firebase Auth, Google Sign-In, and Glide.
Architecture	Added Vertical Slice Architecture and mapped feature slices across backend, web, and mobile.
API Contracts	Updated endpoint list to match current service patterns, including auth refresh, appointments/book, haircut options, posts, comments, reactions, feedback, leave requests, admin stats, and barber income.
Database Diagram	Replaced old relational-style ERD with Firestore collection model and logical relationships.
Acceptance Criteria	Fixed incorrect AC-3 heading and added criteria for login, booking conflicts, leave requests, income records, and admin dashboard.

9.1 Final Design Note

BarberConnect should be presented as a full-stack web/mobile barbershop platform with a Spring Boot REST backend, Firestore NoSQL persistence, Cloudinary media storage, role-based dashboards, and feature-based vertical slices. This wording is the most accurate match for the current repository design.